

Parallel and Distributed Computing (CS3006)

Course Instructor(s):

Dr. Muhammad Arshad Islam, Mr. Farrukh Bashir, Mr.
Aadil ur Rehman, Mr. Fahad Shafique Ali Ashraf,

Section(s): (A, B, C, D, E, F, G H, J, K)

Sessional-I Exam

Total Time (Hrs): 1

Total Marks: 50

Total Questions: 5

Date: Feb 22, 2025

Roll No

Course Section

Student Signature

Do not write below this line.

Attempt all the questions.

[CLO 1: Demonstrate understanding of various concepts involved in parallel and distributed computer architectures.]

Question: Use your understanding of mutexes and critical sections to provide the output of the code. [5 marks]

```
#define NUM_THREADS 4
#define ITERATIONS 2

int shared_counter = 0;
int thread_turn = 3;
pthread_mutex_t lock;
pthread_cond_t cond;

void* thread_function(void* arg) {
    int tid = *((int*)arg);

    for (int i = 0; i < ITERATIONS; i++) {

        pthread_mutex_lock(&lock);
        while (tid != thread_turn) {
            pthread_cond_wait(&cond, &lock);
        }
        shared_counter++;
        printf("Thread %d (Iteration %d): Counter = %d \n", tid, i
+ 1, shared_counter);

        thread_turn = (thread_turn + 1) % NUM_THREADS;
        pthread_cond_broadcast(&cond);

        pthread_mutex_unlock(&lock);
    }

    printf("Thread %d: Completed all iterations.\n", tid);
}
```

National University of Computer and Emerging Sciences

Islamabad Campus

```
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int thread_ids[NUM_THREADS];

    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&cond, NULL);

    for (int i = 0; i < NUM_THREADS; i++) {
        thread_ids[i] = i;
        pthread_create(&threads[i], NULL, thread_function,
&thread_ids[i]);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cond);

    printf("\nFinal Value of shared_counter: %d\n", shared_counter);
    return 0;
}
```

Thread 3 (Iteration 1): Counter = 1
Thread 0 (Iteration 1): Counter = 2
Thread 1 (Iteration 1): Counter = 3
Thread 2 (Iteration 1): Counter = 4
Thread 3 (Iteration 2): Counter = 5
Thread 3: Completed all iterations.
Thread 0 (Iteration 2): Counter = 6
Thread 0: Completed all iterations.
Thread 1 (Iteration 2): Counter = 7
Thread 1: Completed all iterations.
Thread 2 (Iteration 2): Counter = 8
Thread 2: Completed all iterations

Final Value of shared_counter: 8

[CLO 1: Demonstrate understanding of various concepts involved in parallel and distributed computer architectures.]

Q2: Following is the incomplete code of adding two matrices using pthreads. Provide two functions that use block and cyclic strategy to add the matrices. **[5+5 marks]**

```
const int MAX = 400;
const int MAX_THREAD = 4; // Maximum number of threads
```

National University of Computer and Emerging Sciences

Islamabad Campus

```
int matAdd[MAX][MAX]; // to store results
// Function to perform blockwise matrix Addition
void Block_Addition(int matA[][MAX], int matB[][MAX])
{
    int index = step_add++;
    // Each thread computes 1/4th of matrix addition
    int start = index * (MAX / 4);
    int end = (index + 1) * (MAX / 4);
    for (int i = start; i < end; i++) {
        for (int j = 0; j < MAX; j++) {
            matAdd[i][j] = matA[i][j] + matB[i][j];
        }
    }
}

void Cyclic_Addition(int matA[][MAX], int matB[][MAX])
{
    int tid = pthread_self();
    // Each thread processes rows in a cyclic manner
    for (int i = tid; i < MAX; i += NUM_THREADS) {
        for (int j = 0; j < MAX; j++) {
            matAdd[i][j] = matA[i][j] + matB[i][j];
        }
    }
    pthread_exit(NULL);
}

int main() {
    int matA[MAX][MAX], matB[MAX][MAX]; //assume matrices are filled
    thread add_thread[MAX_THREAD];

    for (int i = 0; i < MAX_THREAD; i++)
        add_thread[i] = thread(BlockAddition, matA, matB);
    for (int i = 0; i < MAX_THREAD; i++)
        add_thread[i] = thread(CyclicAddition, matA, matB);
    // Waiting for all threads to finish
    for (int i = 0; i < MAX_THREAD; i++) {
        add_thread[i].join();
        sub_thread[i].join();
    }

    return 0;
}
```

[CLO 1: Demonstrate understanding of various concepts involved in parallel and distributed computer architectures.]

Q3: Provide a AVX512/NEON equivalent of the following function. Assume each integer is 32 bit.

[10 marks]

```
static const int length = 1024*8;
```

National University of Computer and Emerging Sciences

Islamabad Campus

```
static float a[length];
float scalarAverage() {
    float sum = 0.0;
    for (uint32_t j = 0; j < _countof(a); ++j) {
        sum += a[j];
    }
    return sum / _countof(a);
}
float avxAverage () {
    __m256 sumx8 = _mm256_setzero_ps();
    for (uint32_t j = 0; j < _countof(a); j = j + 8) {
        sumx8 = _mm256_add_ps(sumx8, _mm256_loadu_ps(&a[j]));
    }
    float sum = sumx8.m256_f32[0] + sumx8.m256_f32[1] +
        sumx8.m256_f32[2] + sumx8.m256_f32[3] +
        sumx8.m256_f32[4] + sumx8.m256_f32[5] +
        sumx8.m256_f32[6] + sumx8.m256_f32[7];
    return sum / _countof(a);
}
```

[CLO 2: 2. Implement different parallel and distributed programming paradigms and algorithms using Message-Passing Interface (MPI) and OpenMP.]

Q4: Calculate Sum of an array using MPI under following preconditions. [15 marks]

- The size of the array as well as number of processes is unknown at compile time
- Rank=0 will distribute the array among processes
- Use of scatter and gather functions is not allowed.

`int a[] ; // To be summed, assume it is filled`

```
int main(int argc, char* argv[])
{
    int pid, np, elements_per_process, n_elements_recieved;
    // np -> no. of processes
    // pid -> process id

    MPI_Status status;
    MPI_Init(&argc, &argv);

    // find out process ID, and how many processes were started
    MPI_Comm_rank(MPI_COMM_WORLD, &pid);
    MPI_Comm_size(MPI_COMM_WORLD, &np);

    // master process
    if (pid == 0) {
        int index, i;
        elements_per_process = n / np;
        int sum = 0;
        if (np > 1) { // check if more than 1 processes are run
            //distributes the portion of array to child processes to
            //calculate their partial sums
            for (i = 1; i < np - 1; i++) {
                index = i * elements_per_process;
                MPI_Send(&elements_per_process, 1, MPI_INT, i, 0,
                    MPI_COMM_WORLD);
            }
        }
    }
}
```

National University of Computer and Emerging Sciences

Islamabad Campus

```
        MPI_Send(&a[index],elements_per_process,
        MPI_INT, i, 0,MPI_COMM_WORLD);
    }

    // last process adds remaining elements
    index = i * elements_per_process;
    int elements_left = n - index;

    MPI_Send(&elements_left,1, MPI_INT,i, 0,
    MPI_COMM_WORLD);
    MPI_Send(&a[index],elements_left,MPI_INT, i, 0,
    MPI_COMM_WORLD);
}

// master process add its own sub array

for (i = 0; i < elements_per_process; i++)
    sum += a[i];

// collects partial sums from other processes
int tmp;
for (i = 1; i < np; i++) {
    MPI_Recv(&tmp, 1, MPI_INT,MPI_ANY_SOURCE, 0,
    MPI_COMM_WORLD, &status);
    int sender = status.MPI_SOURCE;
    sum += tmp;
}

// prints the final sum of array
printf("Sum of array is : %d\n", sum);
}
else { // slave processes
    MPI_Recv(&n_elements_recieved,1, MPI_INT, 0, 0,
    MPI_COMM_WORLD,&status);

    // stores the received array segment in local array a2
    MPI_Recv(&a2, n_elements_recieved,MPI_INT, 0, 0,
    MPI_COMM_WORLD,&status);

    // calculates its partial sum
    int partial_sum = 0;
    for (int i = 0; i < n_elements_recieved; i++)
        partial_sum += a2[i];

    // sends the partial sum to the root process
    MPI_Send(&partial_sum, 1, MPI_INT,0, 0, MPI_COMM_WORLD);
}
MPI_Finalize();
return 0;
}
```

National University of Computer and Emerging Sciences
Islamabad Campus